

NLP-based Sentiment Analysis: A Deep Learning Approach Using BERT

Kaggle Data Source: [NLP – Sentiment Analysis - XM](#)

Problem Description

Problem Statement

Sentiment analysis is a natural language processing approach to evaluating the emotional context of a language sample expressed as text. This versatile and powerful tool has established applications in a range of fields from customer care to political science. Adopted and modified from the [SMILE Project](#), the goal of this Kaggle project was to use bidirectional encoder representation of transformers (BERT) for categorical assignment of tweets into sentiment groups. The contributors to this project chose this project due to the relevance of sentiment analysis in their specific fields of Public Health and Autism Research.

Dataset Description

The training dataset contains three columns (ID, Text, and Sentiment) for 1600 tweets. The test dataset consisted of 400 tweets with two columns (ID and Text). Distribution of tweet length in the training and test datasets were similar and both roughly normal (*Figure 1*). These distributions illustrate tweet length after removal of usernames and website links, but for the purposes of analyses these elements were retained because preprocessing of tweets did not improve model performance and therefore was omitted. The single instance of tweet length zero in the test dataset was a tweet that contained a smiley face emoji and a username tag. There were no missing data in these datasets.

The Text column for both datasets contain the tweet string of characters including username tags, website links, hashtags, and emojis formed from punctuation. The tweets in these datasets primarily referenced experiences, exhibits, or controversy related to the British Museum.

The sentiment column contained a categorical classification (0-3) into one of four categories. The categories were not defined within the Kaggle project, but qualitative review of a sample of the training dataset suggested that the categories roughly mapped onto (0) positive valence moderate arousal such as happy, (1) negative valence with high arousal such as angry, (2) neutral valence and arousal like an absence of sentiment, and (3) negative valence with low arousal such as sad. The distribution of sentiments across categories was roughly equal (*Figure 2*).

Figure 1. Histogram of Tweet Length in Training and Test Datasets

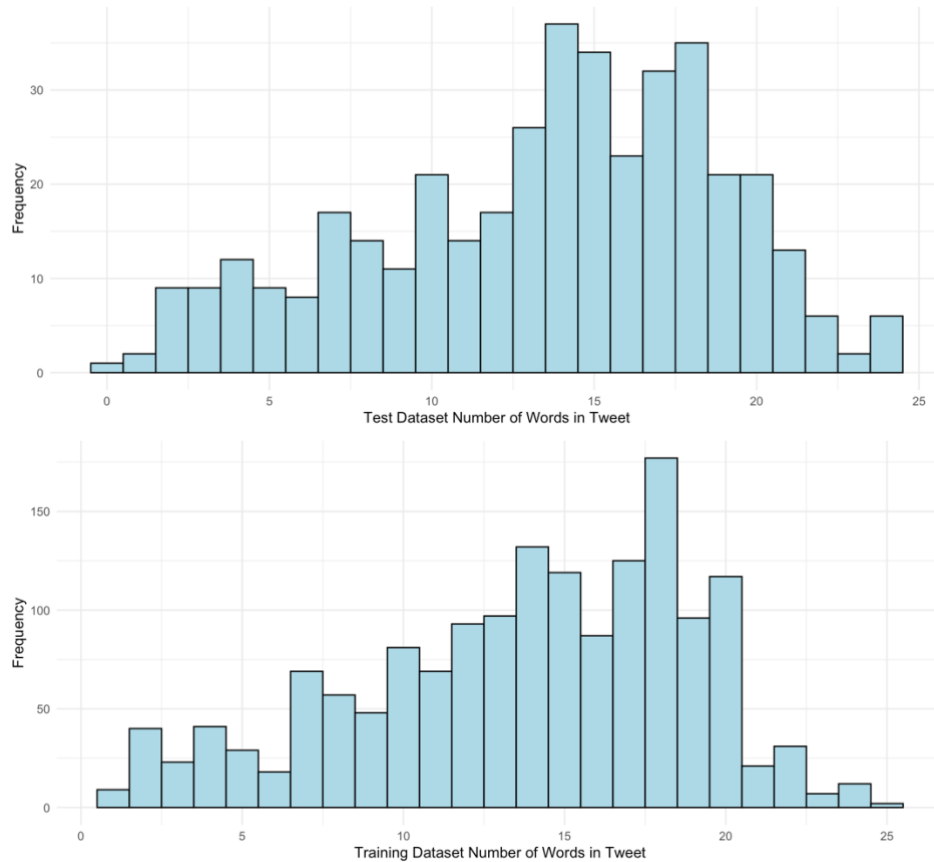
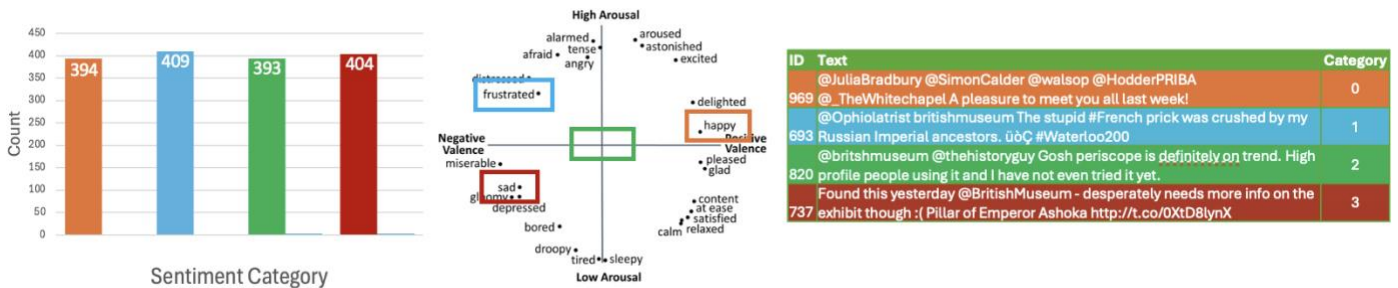


Figure 2. Training Dataset Sentiment Category Distribution and Category Definitions



Objective

The objective of this project was to achieve high sentiment classification accuracy on the test dataset by fine-tuning variants of pre-trained BERT models using the provided training data. The task involved predicting the sentiment category, represented by numerical labels, for each text sample in the test set. Model performance was evaluated based on the accuracy of predicted sentiment categories compared to the true labels.

Relevance to Field

Sentiment analysis typically targets self-contained language samples such as reviews or articles. In contrast, tweets are context-dependent, often responding to events, experiences, or other tweets, resembling conversational turns in discourse. This section examines the relevance of tweet sentiment analysis across disciplines and explores a clinical application involving similarly structured conversational language samples.

Utility of Detecting Emotions through Tweets

Sentiment analysis of Twitter data has emerged as a large-scale and real-time informative approach to understanding public sentiment on current events. Sentiment analysis of twitter data can summarize the tone of huge amounts of data with efficiency. This is valuable in several disciplines:

- Within political science, perception of political campaigns or voter reactions to speeches is an incredible resource intense and time-consuming endeavor. Sentiment analysis of publicly available tweets are an ethical and cost-effective approach to assessing the efficacy of a speech, ad, or policy initiative (Ansari et al., 2020).
- Within the field of transportation, sentiment analysis can be applied in crisis management (Patel et al., 2023). Airlines experiencing outages or delays can tailor their response to the sentiment of their passengers that is publicly expressed online.
- Within the field of psychology and sociology, the perception of mental health diagnoses can be observed through sentiment analysis of tweets containing references to these conditions (Norwahidah Mohamad Yazid, 2024). Additionally, temporal changes in public perception or individual experience of mental health diagnoses can be detected. This is an especially relevant issue when a mental health condition is referenced in the news due to current events or policy changes as the stigma around that condition may change sharply.
- Within the field of medicine, sentiment analysis has been applied to track public perception of emerging crises such as the COVID-19 pandemic (Priyadarshini et al., 2022). Large-scale psychological coping with public health crises can be understood and responses can be developed to address emerging barriers to the uptake of treatment that sentiment analysis detects.

Clinical Applications

Social communication is the use of verbal and non-verbal behaviors to establish, maintain, and enhance relationships (Hansen et al., 2018). Social communication challenges are prevalent across many developmental disorders. Measurement of social communication challenges is necessary for both diagnostic purposes and in order to track the efficacy of novel interventions and other therapeutics (Bishop et al., 2019). In autism specifically, tools in this area are limited to indirect proxy- and self-report surveys such as the Social Communication Questionnaire (SCQ; Rutter et al., 2003) and resource intense clinician observation measures such as the Autism Diagnostic Observation Schedule (ADOS; Lord et al., 2012). These measures require extended in-person appointments and are not broadly scalable or accessible.

Synchrony, alignment, and entrainment have emerged as potential methods for quantifying social communication. These terms refer to the same construct that emerged concurrently in the disciplines of speech science, linguistics, and psychology. For the purposes of this report, the term synchrony will be used to refer to this construct. Synchrony seeks to measure the extent to which interlocutors match or mirror one another within specific domains of social communication. Within the arena of non-verbal social communication, prosodic features of speech such as volume, pitch, and rate have been examined by the field of speech science (Levitan et al., 2012; Michalsky et al., 2018; Pickering & Garrod, 2004; Reitter & Moore, 2014; Wynn et al., 2023). Synchrony quantification methods of other non-verbal features such as gesture, gaze, and facial expression are being developed through machine learning vision analyses of recorded clinician assessments (Avramidis et al., 2024). Additionally, synchrony in several aspects of verbal content including word choice, phrase structure, and topic maintenance is being assessed for both child and adult interlocutors (Duran et al., 2019; Fusaroli et al., 2023b, 2023a). Sentiment analysis of verbal content remains an unaddressed area of synchrony in our understanding of social communication quantification. There are many challenges to sentiment identification in this work including variation specific to child speech samples, inherent missing context in conversational turns between interlocutors, and precision of continuous rather than categorical sentiment estimation. High performance of pre-trained BERT models in this Kaggle Competition is encouraging and indicates that this valuable work will be possible with existing tools.

Methodology

Overview of Approaches

For this project we constructed 7 models using four pre-trained variants of BERT (*Table 1*). The BERT variants used included Base BERT, DistilBERT, RoBERTa, and BERTweet. Base BERT is the original transformer model developed by Google in 2018, trained on large corpora like Wikipedia and BookCorpus using masked language modeling and next sentence prediction (Devlin et al., 2019). DistilBERT is a compressed version of BERT that maintains approximately 97% of its performance while being smaller and faster, due to a training process called knowledge distillation (Sanh et al., 2020). RoBERTa builds on BERT by removing the next sentence prediction objective, employing dynamic masking, and training on a much larger dataset for longer durations, resulting in improved performance on various NLP tasks (Liu et al., 2019). In contrast, BERTweet is a domain-specific variant pre-trained on 850 million English-language tweets, making it particularly well-suited for analyzing the informal language, abbreviations, and symbols common on Twitter (Nguyen et al., 2020). Once BERTweet was established as the variant that is better in performance, model evaluation methods such as 5-fold cross validation and leave-one out cross validation were conducted to analyze performance.

Table 1. BERT Variants and Modeling Approaches for Sentiment Classification

Model #	Model Name	Formula/ Representation	Description
1	BERT (baseline) + PyTorch	$BERT(x) \rightarrow CLS \rightarrow \text{Softmax}(4)$	Standard BERT (12-layer) model for sequence classification using PyTorch and HuggingFace.
2	DistilBERT for Sentiment Classification	$DistilBERT(x) \rightarrow CLS \rightarrow \text{Softmax}(4)$	A smaller, faster version of BERT that retains 95% of its performance with fewer parameters.
3	RoBERTa-Based Sentiment Classification	$RoBERTa(x) \rightarrow CLS \rightarrow \text{Softmax}(4)$	An optimized BERT variant trained longer with more data and dynamic masking (no NSP).
4	BERTweet for Sentiment Classification	$BERTweet(x) \rightarrow CLS \rightarrow \text{Softmax}(4)$	A RoBERTa-style model pre-trained on 850M English tweets; ideal for Twitter-specific sentiment tasks.
5	BERTweet + Scheduler + Early Stopping (WINNING MODEL)	$BERTweet(x) \rightarrow CLS \rightarrow \text{Softmax}(4) + \text{LR Scheduler} + \text{EarlyStopping}$	Same as Model 4 but includes learning rate scheduling and early stopping to prevent overfitting.
6	BERTweet + LOOCV + Hyperparameters + Early Stopping	$\sum \text{BERTweet}_i(x) \text{ over LOOCV folds} \rightarrow CLS \rightarrow \text{Softmax}(4) + \text{Tuned Hyperparams}$	Uses Leave-One-Out Cross-Validation for robust evaluation, plus tuned LR and early stopping.
7	BERTweet + 5-Fold CV + Scheduler + Early Stopping	$BERTweet + 5\text{-Fold CV} + \text{Scheduler} + \text{Early Stopping}$	BERTweet + 5-Fold CV + Scheduler and Early Stopping

Rationale for the Chosen Method

The project required the models to be run on BERT. Of the four BERT variants applied, BERTweet showed the highest accuracy of test dataset sentiment classification. Therefore, we decided to further fine tune this model by with the additions of a learning rate scheduler and early stopping. This improved performance while reducing overfitting and improving generalizability to both the public and the private Kaggle test sets.

Implementation Details and Reproducibility

The implementation was conducted on a Google Colab environment utilizing a T4 GPU. Python scripts (*in the Appendix*) ensure full reproducibility of the process. Several BERT model variants were selected based on their representativeness across the range of available architectures and their suitability for the dataset characteristics.

Model optimization employed a linear learning rate scheduler without a warm-up phase. The learning rate decayed linearly from an initial value of 2×10^{-5} toward zero over the course of training, promoting stable convergence. Each update to the learning rate occurred once per batch. The optimizer used was AdamW, chosen for its effective handling of sparse gradients and its incorporation of L2 regularization decoupled from the learning rate, which is critical for improving generalization in transformer-based models. Training was managed with an early stopping strategy, monitoring validation accuracy at the end of each epoch. If no improvement

was observed across two consecutive epochs (patience=2), training was halted, and the checkpoint corresponding to the highest validation accuracy was preserved for inference.

Hyperparameter tuning was deliberately minimal. Fixed values were selected manually for key parameters such as learning rate, batch size (16), maximum sequence length (128 tokens), and number of epochs (≤ 5), based on prior experimentation on similar medium-sized datasets. Although these values yielded satisfactory results, it is acknowledged that transformer models exhibit high sensitivity to hyperparameters, particularly learning rate, weight decay, warm-up ratio, and sequence length. A comprehensive hyperparameter search could potentially optimize performance further. In the context of the Kaggle competition, sentiment analysis was treated as a single-label categorical classification task. However, research from emotion psychology suggests that sentiment can be more accurately represented in a two-dimensional affective space, with axes corresponding to valence (positive-negative) and arousal (high-low emotional intensity). In a more sophisticated framework, sentiment prediction could involve generating two continuous values per sample, positioning each within this affective space (*Figure 2*).

Given the simplicity and novelty of the task structure and the clarity of class boundaries, model complexity was kept minimal. Techniques such as model ensembling, seed averaging, or complex multi-output architectures were not pursued, as straightforward models already achieved high accuracy and reliable performance metrics.

Team Collaboration

Shilpa and Mags developed the approach together and both explored the documentation for Base BERT. DistilBERT and RoBERTa were proposed by Shilpa and Mags proposed BERTweet. Both drafted code for the first three models, but Shilpa's code version was used for final analyses due to better organization. Mags proposed testing impacts of preprocessing and validation strategies, and Shilpa wrote the code to implement these ideas. Shilpa and Mags reviewed the results together and co-created the presentation that they shared with the class.

Results and Evaluation

Performance Metrics

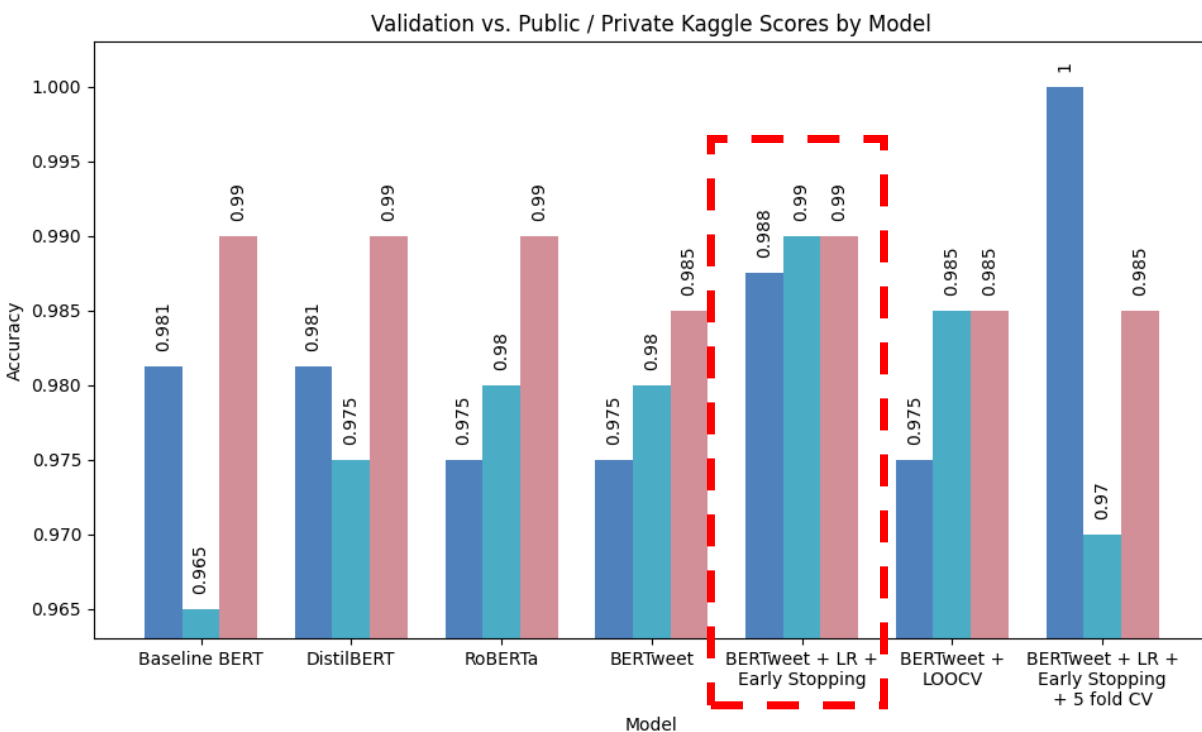
The primary performance metric used to evaluate model performance was classification accuracy, measured on both the public and private Kaggle test datasets, as well as on the internal validation split during model training. Accuracy, defined as the proportion of correct predictions over total predictions, was deemed appropriate for this task because the sentiment classification problem was formulated as a single-label categorical classification, and the distribution of sentiment classes was approximately balanced across categories. Therefore, accuracy provided a direct and interpretable measure of model performance without the need for adjustments such as class weighting or alternative metrics like macro-F1 score. Validation accuracy, public Kaggle score, and private Kaggle score were all recorded for each model variation to assess

generalization performance (Table 2 and Figure 3). These three evaluation points enabled comparison not only within the training data framework but also on unseen external datasets, reducing the likelihood of model overfitting.

Table 2. Accuracy Scores across Model Variants

Model Name	Validation Accuracy Scores	Kaggle Scores	
		Private Score	Public Score
BERT (baseline)	0.981	0.99	0.965
DistilBERT for Sentiment Classification	0.981	0.99	0.975
RoBERTa-Based Sentiment Classification	0.975	0.99	0.98
BERTweet for Sentiment Classification	0.975	0.985	0.98
BERTweet + Scheduler + Early Stopping	0.988	0.99	0.99
BERTweet + LOOCV + Early Stopping	0.975	0.985	0.985
BERTweet + 5-Fold CV + LR + Early Stopping	1	0.985	0.97

Figure 3. Classification Accuracy Scores for Model Variants



Analysis of Results and Comparative Analysis

The winning model configuration, **BERTweet + Learning Rate Scheduler + Early Stopping**, achieved the highest combined performance across validation, public, and private datasets (Validation Accuracy: 0.988, Public Kaggle Score: 0.990, Private Kaggle Score: 0.990). This model effectively balanced high training accuracy with generalization, demonstrating minimal overfitting (despite model 7 achieving a higher validation accuracy, but failing to generalize to other test datasets).

Strengths

- **Domain Adaptation:** BERTweet, being pretrained on Twitter data, captured the informal, abbreviated, and noisy characteristics of tweets more effectively than generic models like baseline BERT, DistilBERT, or RoBERTa
- **Optimization Stability:** The use of a linear learning rate scheduler combined with early stopping prevented overfitting, allowing the model to converge smoothly and terminate training once performance plateaued
- **Reproducibility and Simplicity:** The pipeline involved minimal hyperparameter tuning yet consistently achieved strong results, highlighting the robustness of the chosen approach

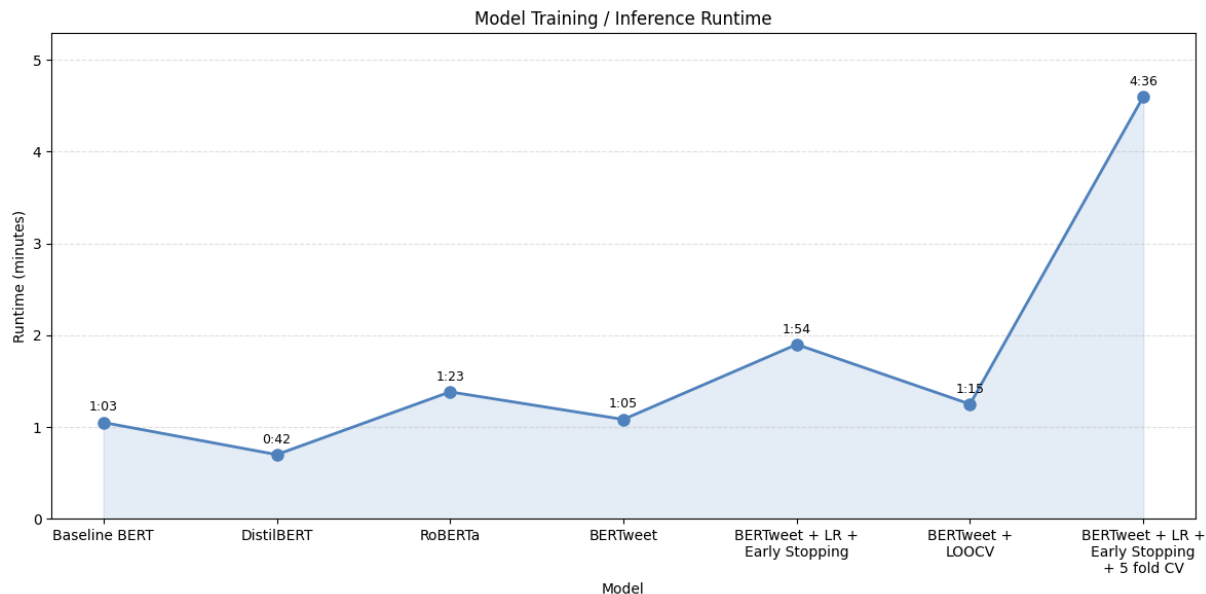
Weaknesses

- **Sensitivity to Hyperparameters:** Despite reasonable defaults, performance could potentially be further optimized through systematic hyperparameter tuning (e.g., tuning warm-up steps, weight decay)
- **Model Interpretability:** Transformer models, including BERTweet, lack inherent interpretability, and no attention visualization or feature importance analysis was conducted to limit the project scope

Interesting Findings

- **Minimal Gains from Cross-Validation:** Additional cross-validation strategies, such as leave-one-out cross-validation (LOOCV) and 5-fold cross-validation, did not yield substantial improvements over simple validation splits. In fact, the BERTweet + 5-Fold CV model showed a slight decrease in Kaggle scores despite achieving a perfect internal validation accuracy of 1.0. This suggests that, for relatively small and well-behaved datasets, simpler validation strategies are sufficient and more computationally efficient
- **Runtime Trade-offs:** Real time sentiment classification of tweets is a meaningful source of monitoring in public health. The four BERT variants exhibited high efficiency in this moderately sized dataset and ran in under 1.5 minutes. However, hyperparameter tuning extended the runtime slightly for model 5. More complex validation strategies (e.g., 5-Fold CV) resulted in disproportionately higher runtimes (up to 4 minutes and 36 seconds) compared to basic models (of under 1.5 minutes). However, the performance improvement did not justify the additional computational cost and time (*Figure 4*).

Figure 4. Model Runtime



Limitations and Future Directions

This work involved limited hyperparameter tuning, relying on hand-selected settings without broader searches, which could have uncovered better combinations. Validation was based on a single 90/10 split, making results sensitive to data variance; future work should adopt k-fold cross-validation with multiple seeds for greater stability. The small, noisy dataset likely capped achievable performance and introduced label ambiguity, suggesting that larger, cleaner datasets or augmentation strategies could enhance generalization. Model exploration was restricted to smaller variants due to computational limits; scaling to larger architectures like BERTweet-large or DeBERTa-large could offer gains. Lastly, no ensembling or seed averaging was applied, and incorporating these techniques could further improve robustness and final model accuracy.

Team Collaboration

The proposal, coding, and evaluation iterations were very cooperative. Together, Shilpa and Mags examined the model's outputs, evaluated the effects of different experimental modifications, and came to well-informed conclusions based on their shared interpretation of the data. At every stage of the project, this collaborative structure improved critical evaluation, guaranteeing that decisions were well-rounded, supported by data, and methodically verified. In the end, it improved the project's conclusions' clarity and the final model selection's resilience.

Appendix

References

- Ansari, M. Z., Aziz, M. B., Siddiqui, M. O., Mehra, H., & Singh, K. P. (2020). Analysis of Political Sentiment Orientations on Twitter. *Procedia Computer Science*, 167, 1821–1828. <https://doi.org/10.1016/j.procs.2020.03.201>
- Avramidis, K., Chang, M. Y., Sharma, R., Borchert, M. S., & Narayanan, S. (2024). *Evaluating Atypical Gaze Patterns through Vision Models: The Case of Cortical Visual Impairment* (arXiv:2402.09655). arXiv. <https://doi.org/10.48550/arXiv.2402.09655>
- Bishop, S., Farmer, C., Kaat, A., Georgiades, S., Kanne, S., & Thurm, A. (2019). The need for a developmentally based measure of social-communication skills. *Journal of the American Academy of Child and Adolescent Psychiatry*, 58(6), 555–560. <https://doi.org/10.1016/j.jaac.2018.12.010>
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding* (arXiv:1810.04805). arXiv. <https://doi.org/10.48550/arXiv.1810.04805>
- Duran, N. D., Paxton, A., & Fusaroli, R. (2019). ALIGN: Analyzing linguistic interactions with generalizable techNiques—A Python library. *Psychological Methods*, 24(4), 419–438. <https://doi.org/10.1037/met0000206>
- Fusaroli, R., Weed, E., Rocca, R., Fein, D., & Naigles, L. (2023a). Caregiver linguistic alignment to autistic and typically developing children: A natural language processing approach illuminates the interactive components of language development. *Cognition*, 236, 105422. <https://doi.org/10.1016/j.cognition.2023.105422>
- Fusaroli, R., Weed, E., Rocca, R., Fein, D., & Naigles, L. (2023b). Repeat after me? Both children with and without Autism commonly align their language with that of their caregivers. *Cognitive Science*, 47(11), e13369. <https://doi.org/10.1111/cogs.13369>
- Hansen, S. G., Carnett, A., & Tullis, C. A. (2018). Defining Early Social Communication Skills: A Systematic Review and Analysis. *Advances in Neurodevelopmental Disorders*, 2(1), 116–128. <https://doi.org/10.1007/s41252-018-0057-5>
- Levitan, R., Gravano, A., Willson, L., Beňuš, S., Hirschberg, J., & Nenkova, A. (2012). Acoustic-Prosodic Entrainment and Social Behavior. In E. Fosler-Lussier, E. Riloff, & S. Bangalore (Eds.), *Proceedings of the 2012 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* (pp. 11–19). Association for Computational Linguistics. <https://aclanthology.org/N12-1002>
- Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., & Stoyanov, V. (2019). *RoBERTa: A Robustly Optimized BERT Pretraining Approach* (arXiv:1907.11692). arXiv. <https://doi.org/10.48550/arXiv.1907.11692>
- Lord, C., Luyster, R. J., Gotham, K., & Guthrie, W. (2012). *Autism diagnostic observation schedule, second edition (ADOS-2) manual*. Western Psychological Services Torrance, CA.
- Michalsky, J., Schoormann, H., & Niebuhr, O. (2018). Conversational quality is affected by and reflected in prosodic entrainment: 9th International Conference on Speech Prosody 2018. *Proceedings of the 9th International Conference on Speech Prosody 2018*, 389–392. <https://doi.org/10.21437/SpeechProsody.2018-79>
- Nguyen, D. Q., Vu, T., & Nguyen, A. T. (2020). *BERTweet: A pre-trained language model for English Tweets* (arXiv:2005.10200). arXiv. <https://doi.org/10.48550/arXiv.2005.10200>
- Norwahidah Mohamad Yazid. (2024). A Sentiment Analysis of Autism Tweets. *Journal of Electrical Systems*, 20(4s), 582–589. <https://doi.org/10.52783/jes.2074>
- Patel, A., Oza, P., & Agrawal, S. (2023). Sentiment Analysis of Customer Feedback and Reviews for Airline Services using Language Representation Model. *Procedia Computer Science*, 218, 2459–2467. <https://doi.org/10.1016/j.procs.2023.01.221>
- Pickering, M. J., & Garrod, S. (2004). Toward a mechanistic psychology of dialogue. *Behavioral and Brain Sciences*, 27(2), 169–226. <https://doi.org/10.1017/S0140525X04000056>
- Priyadarshini, I., Mohanty, P., Kumar, R., Sharma, R., Puri, V., & Singh, P. K. (2022). A study on the sentiments and psychology of twitter users during COVID-19 lockdown period. *Multimedia Tools and Applications*, 81(19), 27009–27031. <https://doi.org/10.1007/s11042-021-11004-w>
- Reitter, D., & Moore, J. D. (2014). Alignment and task success in spoken dialogue. *Journal of Memory and Language*, 76, 29–46. <https://doi.org/10.1016/j.jml.2014.05.008>
- Rutter, M., Bailey, A., & Lord, C. (2003). *Manual for the Social Communication Questionnaire*. Western Psychological Services Los Angeles, CA.
- Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2020). *DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter* (arXiv:1910.01108). arXiv. <https://doi.org/10.48550/arXiv.1910.01108>
- Wynn, C. J., Barrett, T. S., Berisha, V., Liss, J. M., & Borrie, S. A. (2023). Speech Entrainment in Adolescent Conversations: A Developmental Perspective. *Journal of Speech, Language, and Hearing Research*, 66(8S), 3132–3150. https://doi.org/10.1044/2023_JSLHR-22-00263

Complete Reproducible Script for All Models and Figures

```
# Import Libraries
import os
import re
import torch
import random
import textwrap
import numpy as np
import pandas as pd
import torch.nn as nn
from tqdm import tqdm
from torch.optim import AdamW
from google.colab import files
import matplotlib.pyplot as plt
from sklearn.metrics import accuracy_score
from torch.utils.data import Dataset, DataLoader
from sklearn.model_selection import StratifiedKFold
from sklearn.model_selection import train_test_split
from transformers import BertTokenizer, BertForSequenceClassification
from transformers import AutoTokenizer, AutoModelForSequenceClassification
from transformers import RobertaTokenizer, RobertaForSequenceClassification
from transformers import DistilBertTokenizer, DistilBertForSequenceClassification
from transformers import AutoTokenizer, AutoModelForSequenceClassification, get_linear_schedule_with_warmup
!pip install -q openpyxl
!pip install transformers datasets torch torchvision torchaudio

# Load Data sets
train_df = pd.read_csv("/content/train.csv")
test_df = pd.read_csv("/content/test_features.csv")
sample_submission_df = pd.read_csv("/content/sample_submission.csv")

# Display a preview of the data
train_preview = train_df.head()
test_preview = test_df.head()
sample_submission_preview = sample_submission_df.head()

train_preview, test_preview, sample_submission_preview

#####
# Model 1: BERT using Hugging Face + PyTorch (baseline model)
#####

# Split into train/validation
train_texts, val_texts, train_labels, val_labels = train_test_split(
    train_df['Text'], train_df['Category'], test_size=0.1, random_state=42
)

# Load BERT tokenizer
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')

# Tokenize training and validation data
train_encodings = tokenizer(list(train_texts), truncation=True, padding=True, max_length=128)
val_encodings = tokenizer(list(val_texts), truncation=True, padding=True, max_length=128)
test_encodings = tokenizer(list(test_df['Text']), truncation=True, padding=True, max_length=128)

# Create Torch Dataset Class
class SentimentDataset(Dataset):
    def __init__(self, encodings, labels=None):
        self.encodings = encodings
        self.labels = labels

    def __len__(self):
        return len(self.encodings['input_ids'])

    def __getitem__(self, idx):
        item = {key: torch.tensor(val[idx]) for key, val in self.encodings.items()}
        if self.labels is not None:
            item['labels'] = torch.tensor(self.labels[idx])
        return item

# Create datasets
train_dataset = SentimentDataset(train_encodings, list(train_labels))
val_dataset = SentimentDataset(val_encodings, list(val_labels))
test_dataset = SentimentDataset(test_encodings)

# Load Pretrained BERT model
# Load model with 4 output labels (0-3)
model = BertForSequenceClassification.from_pretrained('bert-base-uncased', num_labels=4)
device = torch.device('cuda') if torch.cuda.is_available() else torch.device('cpu')
model.to(device)

# Train the model
train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=16)

# Initialize optimizer from torch.optim
optim = AdamW(model.parameters(), lr=5e-5)

# Training loop
epochs = 3
for epoch in range(epochs):
    model.train()
    loop = tqdm(train_loader, leave=True)
    for batch in loop:
        batch = {k: v.to(device) for k, v in batch.items()}
        outputs = model(**batch)
        loss = outputs.loss
        loss.backward()
        optim.step()
        optim.zero_grad()
        loop.set_description(f'Epoch {epoch}')
        loop.set_postfix(loss=loss.item())

# Evaluation on Validation data set
model.eval()
preds, true = [], []

with torch.no_grad():
```

```
for batch in val_loader:
    batch = {k: v.to(device) for k, v in batch.items()}
    outputs = model(**batch)
    logits = outputs.logits
    preds.extend(torch.argmax(logits, axis=1).cpu().numpy())
    true.extend(batch['labels'].cpu().numpy())

print("Validation Accuracy:", accuracy_score(true, preds))

#Predict on test set
test_loader = DataLoader(test_dataset, batch_size=16)
test_preds = []

model.eval()
with torch.no_grad():
    for batch in test_loader:
        batch = {k: v.to(device) for k, v in batch.items()}
        outputs = model(**batch)
        logits = outputs.logits
        test_preds.extend(torch.argmax(logits, axis=1).cpu().numpy())

#Prepare submission file
submission = pd.DataFrame({
    'ID': test_df['ID'],
    'Prediction': test_preds
})
submission.to_csv("submission1.csv", index=False)

# Download it
files.download("submission1.csv")

#####
# Model 2: DistilBERT for Sentiment Classification
#####

#Prepare and tokenize
train_texts, val_texts, train_labels, val_labels = train_test_split(
    train_df['Text'], train_df['Category'], test_size=0.1, random_state=42
)

tokenizer = DistilBertTokenizer.from_pretrained('distilbert-base-uncased')

train_encodings = tokenizer(list(train_texts), truncation=True, padding=True, max_length=128)
val_encodings = tokenizer(list(val_texts), truncation=True, padding=True, max_length=128)
test_encodings = tokenizer(list(test_df['Text']), truncation=True, padding=True, max_length=128)

#Create dataset class
class SentimentDataset(Dataset):
    def __init__(self, encodings, labels=None):
        self.encodings = encodings
        self.labels = labels

    def __len__(self):
        return len(self.encodings['input_ids'])

    def __getitem__(self, idx):
        item = {k: torch.tensor(v[idx]) for k, v in self.encodings.items()}
        if self.labels is not None:
            item['labels'] = torch.tensor(self.labels[idx])
        return item

train_dataset = SentimentDataset(train_encodings, list(train_labels))
val_dataset = SentimentDataset(val_encodings, list(val_labels))
test_dataset = SentimentDataset(test_encodings)

#Load DistilBERT model
model = DistilBertForSequenceClassification.from_pretrained(
    'distilbert-base-uncased', num_labels=4
)

device = torch.device('cuda') if torch.cuda.is_available() else torch.device('cpu')
model.to(device)

#Train the model
train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=16)

optimizer = AdamW(model.parameters(), lr=5e-5)

epochs = 3
for epoch in range(epochs):
    model.train()
    loop = tqdm(train_loader, leave=True)
    for batch in loop:
        batch = {k: v.to(device) for k, v in batch.items()}
        outputs = model(**batch)
        loss = outputs.loss
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()
        loop.set_description(f"Epoch {epoch}")
        loop.set_postfix(loss=loss.item())

#Evaluation on validation dataset
model.eval()
val_preds, val_labels_true = [], []

with torch.no_grad():
    for batch in val_loader:
        batch = {k: v.to(device) for k, v in batch.items()}
        outputs = model(**batch)
        logits = outputs.logits
        val_preds.extend(torch.argmax(logits, axis=1).cpu().numpy())
        val_labels_true.extend(batch['labels'].cpu().numpy())

print("Validation Accuracy:", accuracy_score(val_labels_true, val_preds))

#Predict on test set
test_loader = DataLoader(test_dataset, batch_size=16)
```

```
test_preds = []

model.eval()
with torch.no_grad():
    for batch in test_loader:
        batch = {k: v.to(device) for k, v in batch.items()}
        outputs = model(**batch)
        logits = outputs.logits
        test_preds.extend(torch.argmax(logits, axis=1).cpu().numpy())

#Create submission file
submission = pd.DataFrame({
    'ID': test_df['ID'],
    'Prediction': test_preds
})

submission.to_csv("submission2.csv", index=False)
files.download("submission2.csv")

#####
# Model 3: RoBERTa-Based Sentiment Classification
#####

# === TOKENIZATION === #
tokenizer = RobertaTokenizer.from_pretrained('roberta-base')

train_encodings = tokenizer(list(train_texts), truncation=True, padding=True, max_length=128)
val_encodings = tokenizer(list(val_texts), truncation=True, padding=True, max_length=128)
test_encodings = tokenizer(list(test_df['Text']), truncation=True, padding=True, max_length=128)

# === CUSTOM DATASET === #
class SentimentDataset(Dataset):
    def __init__(self, encodings, labels=None):
        self.encodings = encodings
        self.labels = labels

    def __len__(self):
        return len(self.encodings['input_ids'])

    def __getitem__(self, idx):
        item = {key: torch.tensor(val[idx]) for key, val in self.encodings.items()}
        if self.labels is not None:
            item['labels'] = torch.tensor(self.labels[idx])
        return item

train_dataset = SentimentDataset(train_encodings, list(train_labels))
val_dataset = SentimentDataset(val_encodings, list(val_labels))
test_dataset = SentimentDataset(test_encodings)

# === LOAD MODEL === #
model = RobertaForSequenceClassification.from_pretrained('roberta-base', num_labels=4)
device = torch.device('cuda') if torch.cuda.is_available() else torch.device('cpu')
model.to(device)

# === TRAINING SETUP === #
train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=16)
optimizer = AdamW(model.parameters(), lr=2e-5)

# === TRAIN LOOP === #
epochs = 3
for epoch in range(epochs):
    model.train()
    loop = tqdm(train_loader, leave=True)
    for batch in loop:
        batch = {k: v.to(device) for k, v in batch.items()}
        outputs = model(**batch)
        loss = outputs.loss
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()
        loop.set_description(f"Epoch {epoch}")
        loop.set_postfix(loss=loss.item())

# === VALIDATION === #
model.eval()
val_preds, val_true = [], []

with torch.no_grad():
    for batch in val_loader:
        batch = {k: v.to(device) for k, v in batch.items()}
        outputs = model(**batch)
        logits = outputs.logits
        val_preds.extend(torch.argmax(logits, axis=1).cpu().numpy())
        val_true.extend(batch['labels'].cpu().numpy())

acc = accuracy_score(val_true, val_preds)
print(f"Validation Accuracy: {acc:.4f}")

# === PREDICT TEST SET === #
test_loader = DataLoader(test_dataset, batch_size=16)
test_preds = []

model.eval()
with torch.no_grad():
    for batch in test_loader:
        batch = {k: v.to(device) for k, v in batch.items()}
        outputs = model(**batch)
        logits = outputs.logits
        test_preds.extend(torch.argmax(logits, axis=1).cpu().numpy())

# === CREATE SUBMISSION FILE === #
submission = pd.DataFrame({
    'ID': test_df['ID'],
    'Prediction': test_preds
})

submission.to_csv("submission3.csv", index=False)
```

```
### DOWNLOAD SUBMISSION FILE ### #
files.download("submission3.csv")

#####
# Model 4: BERTweet for Sentiment Classification
#####

# Train-Validation Split
train_texts, val_texts, train_labels, val_labels = train_test_split(
    train_df["Text"],
    train_df["Category"],
    test_size=0.1,
    random_state=42
)

# Load BERTweet Tokenizer
# BERTweet model name from Hugging Face: "vinai/bertweet-base"
# This is a RoBERTa-like model pretrained on tweets.
tokenizer = AutoTokenizer.from_pretrained("vinai/bertweet-base")

# Tokenize
train_encodings = tokenizer(list(train_texts), truncation=True, padding=True, max_length=128)
val_encodings = tokenizer(list(val_texts), truncation=True, padding=True, max_length=128)
test_encodings = tokenizer(list(test_df["Text"]), truncation=True, padding=True, max_length=128)

# Create a Custom Dataset Class
class BERTweetDataset(Dataset):
    def __init__(self, encodings, labels=None):
        self.encodings = encodings
        self.labels = labels

    def __len__(self):
        return len(self.encodings["input_ids"])

    def __getitem__(self, idx):
        item = {key: torch.tensor(val[idx]) for key, val in self.encodings.items()}
        if self.labels is not None:
            item["labels"] = torch.tensor(self.labels[idx])
        return item

train_dataset = BERTweetDataset(train_encodings, list(train_labels))
val_dataset = BERTweetDataset(val_encodings, list(val_labels))
test_dataset = BERTweetDataset(test_encodings)

# Load BERTweet Model for Sequence Classification
# Set num_labels=4 to match your classification categories.
model = AutoModelForSequenceClassification.from_pretrained("vinai/bertweet-base", num_labels=4)

# Move model to GPU if available
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)

# Create DataLoaders
train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=16, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=16, shuffle=False)

# Set Up Optimizer
optimizer = AdamW(model.parameters(), lr=5e-5)

# Training Loop
epochs = 3
for epoch in range(epochs):
    model.train()
    loop = tqdm(train_loader, leave=True)
    for batch in loop:
        batch = {k: v.to(device) for k, v in batch.items()}
        outputs = model(**batch)
        loss = outputs.loss
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()

        loop.set_description(f"Epoch {epoch}")
        loop.set_postfix(loss=loss.item())

# Validation
model.eval()
val_preds = []
val_true = []

with torch.no_grad():
    for batch in val_loader:
        batch = {k: v.to(device) for k, v in batch.items()}
        outputs = model(**batch)
        logits = outputs.logits
        val_preds.extend(torch.argmax(logits, axis=1).cpu().numpy())
        val_true.extend(batch["labels"].cpu().numpy())

val_accuracy = accuracy_score(val_true, val_preds)
print("Validation Accuracy:", val_accuracy)

# Predict on Test Set
model.eval()
test_preds = []

with torch.no_grad():
    for batch in test_loader:
```

```
for batch in test_loader:
    batch = {k: v.to(device) for k, v in batch.items()}
    outputs = model(**batch)
    logits = outputs.logits
    test_preds.extend(torch.argmax(logits, axis=1).cpu().numpy())

# Create Submission File
submission = pd.DataFrame({
    "ID": test_df["ID"],
    "Prediction": test_preds
})

submission_filename = "submission4.csv"
submission.to_csv(submission_filename, index=False)
print(f"Created {submission_filename}")

# Download the file
files.download(submission_filename)

#####
# Model 5: BERTweet for Sentiment Classification with Scheduler & Early Stopping (Tuned)
#####

# Set Random Seeds for Reproducibility
def set_seed(seed_value=42):
    random.seed(seed_value)
    np.random.seed(seed_value)
    torch.manual_seed(seed_value)
    torch.cuda.manual_seed_all(seed_value)

set_seed(42)

# Train-Validation Split
train_texts, val_texts, train_labels, val_labels = train_test_split(
    train_df["Text"],
    train_df["Category"],
    test_size=0.1,
    random_state=42
)

# Load BERTweet Tokenizer
tokenizer = AutoTokenizer.from_pretrained("vinai/bertweet-base")

# Tokenize text
train_encodings = tokenizer(
    list(train_texts), truncation=True, padding=True, max_length=128
)
val_encodings = tokenizer(
    list(val_texts), truncation=True, padding=True, max_length=128
)
test_encodings = tokenizer(
    list(test_df["Text"]), truncation=True, padding=True, max_length=128
)

# Create Custom Dataset
class BERTweetDataset(Dataset):
    def __init__(self, encodings, labels=None):
        self.encodings = encodings
        self.labels = labels

    def __len__(self):
        return len(self.encodings["input_ids"])

    def __getitem__(self, idx):
        item = {
            key: torch.tensor(val[idx]) for key, val in self.encodings.items()
        }
        if self.labels is not None:
            item["labels"] = torch.tensor(self.labels[idx])
        return item

train_dataset = BERTweetDataset(train_encodings, list(train_labels))
val_dataset = BERTweetDataset(val_encodings, list(val_labels))
test_dataset = BERTweetDataset(test_encodings)

# Load Model
model = AutoModelForSequenceClassification.from_pretrained(
    "vinai/bertweet-base",
    num_labels=4
)

# Move model to GPU if available
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)

# Create DataLoaders
train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=16, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=16, shuffle=False)

# Optimizer and Learning Rate Scheduler
optimizer = AdamW(model.parameters(), lr=2e-5)

epochs = 5
num_training_steps = epochs * len(train_loader)

scheduler = get_linear_schedule_with_warmup(
    optimizer,
    num_warmup_steps=0,
    num_training_steps=num_training_steps
)

# Early Stopping Setup
best_val_accuracy = 0
early_stop_patience = 2
```

```
epochs_no_improve = 0
best_model_path = "best_model.pt"

# Training Loop with Validation & Early Stopping
for epoch in range(epochs):
    model.train()
    loop = tqdm(train_loader, desc=f"Epoch {epoch}", leave=True)
    for batch in loop:
        optimizer.zero_grad()
        batch = {k: v.to(device) for k, v in batch.items()}
        outputs = model(**batch)
        loss = outputs.loss
        loss.backward()
        optimizer.step()
        scheduler.step()

        loop.set_postfix(loss=loss.item())

    # === Validate after each epoch ===
    model.eval()
    val_preds = []
    val_true = []
    with torch.no_grad():
        for batch in val_loader:
            batch = {k: v.to(device) for k, v in batch.items()}
            outputs = model(**batch)
            logits = outputs.logits
            val_preds.extend(torch.argmax(logits, axis=1).cpu().numpy())
            val_true.extend(batch["labels"].cpu().numpy())

    val_accuracy = accuracy_score(val_true, val_preds)
    print(f"[Epoch {epoch}] Validation Accuracy: {val_accuracy:.4f}")

    # === Early Stopping Logic ===
    if val_accuracy > best_val_accuracy:
        best_val_accuracy = val_accuracy
        epochs_no_improve = 0
        # Save the best model so far
        torch.save(model.state_dict(), best_model_path)
    else:
        epochs_no_improve += 1
        if epochs_no_improve >= early_stop_patience:
            print("Early stopping triggered!")
            break

# === Load the best model weights before inference ===
model.load_state_dict(torch.load(best_model_path))
model.eval()

# Predict on Test Set
test_preds = []
with torch.no_grad():
    for batch in test_loader:
        batch = {k: v.to(device) for k, v in batch.items()}
        outputs = model(**batch)
        logits = outputs.logits
        test_preds.extend(torch.argmax(logits, axis=1).cpu().numpy())

# Create Submission File
submission = pd.DataFrame({
    "ID": test_df["ID"],
    "Prediction": test_preds
})

submission_filename = "submission5_tuned.csv"
submission.to_csv(submission_filename, index=False)
print(f"Created {submission_filename}")

# Download the file if using Colab
files.download(submission_filename)

#####
# Model 6: BERTweet for Sentiment Classification with LOOCV, advanced hyperparameters, and early stopping
#####

# Set Random Seeds for Reproducibility
def set_seed(seed_value=42):
    random.seed(seed_value)
    np.random.seed(seed_value)
    torch.manual_seed(seed_value)
    torch.cuda.manual_seed_all(seed_value)

set_seed(42)

# Basic Text Preprocessing Function
def basic_preprocess(text):
    """
    Example function to remove URLs, user mentions,
    and handle repeated punctuation or other artifacts.
    Modify as needed for your data.
    """
    # Remove URLs
    text = re.sub(r"http\S+|www\S+", "", text)
    # Remove user mentions
    text = re.sub(r"@w+", "", text)
    # Optionally remove extra punctuation or repeated characters, e.g. '!!!!'
    text = re.sub(r"([!?,])\1+", r"\1", text)
    # Strip extra whitespace
    text = text.strip()
    return text

# LOOCV Function
def loocv_bertweet(train_data, labels, num_labels=4,
    epochs=2, lr=2e-5, max_length=128):
    """
    Perform Leave-One-Out Cross-Validation (LOOCV) using BERTweet.

    Args:
        train_data: list of texts
    """
```



```
labels: list of sentiment categories
num_labels: number of output classes
epochs: number of epochs for each LOOCV iteration
lr: learning rate
max_length: tokenizer max_length

Returns:
    overall_accuracy (float): LOOCV accuracy across all training samples
    predictions (list): model predictions for each data point in LOOCV

"""
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
tokenizer = AutoTokenizer.from_pretrained("vinai/bertweet-base")

loocv_preds = []
loocv_true = []

n_samples = len(train_data)
print(f"Starting LOOCV with {n_samples} samples...")

for i in range(n_samples):
    # 1. Build train/val subsets for this iteration
    val_text = [train_data[i]]
    val_label = [labels[i]]
    # Everything except the i-th sample is training
    train_texts = train_data[:i] + train_data[i+1:]
    train_lbs = labels[:i] + labels[i+1:]

    # 2. Tokenize
    train_encodings = tokenizer(
        train_texts, truncation=True, padding=True, max_length=max_length
    )
    val_encodings = tokenizer(
        val_text, truncation=True, padding=True, max_length=max_length
    )

    # 3. Create Datasets
    tr_dataset = BERTweetDataset(train_encodings, train_lbs)
    va_dataset = BERTweetDataset(val_encodings, val_label)

    # 4. Create DataLoaders
    train_loader = DataLoader(tr_dataset, batch_size=16, shuffle=True)
    val_loader = DataLoader(va_dataset, batch_size=1, shuffle=False)

    # 5. Model, Optimizer, Scheduler
    model = AutoModelForSequenceClassification.from_pretrained(
        "vinai/bertweet-base",
        num_labels=num_labels
    )
    model.to(device)
    optimizer = AdamW(model.parameters(), lr=lr)
    total_steps = epochs * len(train_loader)
    scheduler = get_linear_schedule_with_warmup(
        optimizer,
        num_warmup_steps=0,
        num_training_steps=total_steps
    )

    # 6. Train for the specified number of epochs
    for epoch in range(epochs):
        model.train()
        loop = tqdm(train_loader, desc=f"LOOCV Sample {i} | Epoch {epoch}", leave=False)
        for batch in loop:
            batch = {k: v.to(device) for k, v in batch.items()}
            outputs = model(**batch)
            loss = outputs.loss
            loss.backward()
            optimizer.step()
            scheduler.step()
            optimizer.zero_grad()

    # 7. Evaluate on the single validation sample
    model.eval()
    pred_label = None
    with torch.no_grad():
        for batch in val_loader:
            batch = {k: v.to(device) for k, v in batch.items()}
            outputs = model(**batch)
            logits = outputs.logits
            pred_label = torch.argmax(logits, axis=1).cpu().numpy()[0]

    loocv_preds.append(pred_label)
    loocv_true.append(val_label[0])

# Compute overall LOOCV accuracy
loocv_accuracy = accuracy_score(loocv_true, loocv_preds)
print(f"LOOCV Accuracy: {loocv_accuracy:.4f}")

return loocv_accuracy, loocv_preds

#BERTweet Dataset Class
class BERTweetDataset(Dataset):
    def __init__(self, encodings, labels=None):
        self.encodings = encodings
        self.labels = labels

    def __len__(self):
        return len(self.encodings["input_ids"])

    def __getitem__(self, idx):
        item = {key: torch.tensor(val[idx]) for key, val in self.encodings.items()}
        if self.labels is not None:
            item["labels"] = torch.tensor(self.labels[idx])
        return item

# Perform LOOCV
# Convert your data to lists
all_texts = train_df["Text"].tolist()
all_labels = train_df["Category"].tolist()

#Final Model Training on the Entire Training Set + Test Predictions
```

```
# - With advanced hyperparameters, early stopping, etc.

def train_final_model_and_predict(
    train_texts, train_labels, test_texts,
    lr=2e-5, epochs=5, max_length=128,
    early_stop_patience=2
):
    """
    Train a BERTweet model on the entire training set using
    advanced hyperparameters and early stopping, then predict on the test set.
    """
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    tokenizer = AutoTokenizer.from_pretrained("vinai/bertweet-base")

    # Tokenize
    train_encodings = tokenizer(train_texts, truncation=True, padding=True, max_length=max_length)
    test_encodings = tokenizer(test_texts, truncation=True, padding=True, max_length=max_length)

    # Simple train/val split from the training data
    tr_texts, val_texts, tr_labels, val_labels = train_test_split(
        train_encodings["input_ids"],
        train_labels,
        test_size=0.1,
        random_state=42
    )

    # For attention_mask, etc., we need to do the same split
    def split_dict_encodings(encodings, idxs):
        return {k: [encodings[k][i] for i in idxs] for k in encodings}

    # Indices for the newly created train/val sets
    train_indices = list(range(len(tr_texts)))
    val_indices = list(range(len(val_texts)))

    # Rebuild subsets from the main train encodings
    train_subenc = {
        "input_ids": tr_texts,
        "attention_mask": [train_encodings["attention_mask"][i]
                           for i in range(len(train_encodings["input_ids"]))
                           if train_encodings["input_ids"][i] in tr_texts] # simplistic approach
    }
    # The above approach is simplistic. A more rigorous approach is to keep track
    # of the indices from train_test_split. Let's do that properly below.

    # Instead, let's do a single pass approach:
    # (1) Build an index array of the entire dataset
    full_idx = np.arange(len(train_encodings["input_ids"]))
    # (2) Split it
    idx_tr, idx_val = train_test_split(full_idx, test_size=0.1, random_state=42)

    # (3) Slice
    def slice_encodings(enc, indices):
        return {k: [enc[k][i] for i in indices] for k in enc}

    train_enc_sliced = slice_encodings(train_encodings, idx_tr)
    val_enc_sliced = slice_encodings(train_encodings, idx_val)
    tr_labels_sliced = [train_labels[i] for i in idx_tr]
    val_labels_sliced = [train_labels[i] for i in idx_val]

    # Create dataset objects
    train_dataset = BERTweetDataset(train_enc_sliced, tr_labels_sliced)
    val_dataset = BERTweetDataset(val_enc_sliced, val_labels_sliced)
    test_dataset = BERTweetDataset(test_encodings)

    # DataLoaders
    train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
    val_loader = DataLoader(val_dataset, batch_size=16, shuffle=False)
    test_loader = DataLoader(test_dataset, batch_size=16, shuffle=False)

    # Model, optimizer, scheduler
    model = AutoModelForSequenceClassification.from_pretrained("vinai/bertweet-base", num_labels=4)
    model.to(device)
    optimizer = AdamW(model.parameters(), lr=lr)
    num_training_steps = epochs * len(train_loader)
    scheduler = get_linear_schedule_with_warmup(optimizer, num_warmup_steps=0, num_training_steps=num_training_steps)

    # Early Stopping Setup
    best_val_accuracy = 0
    epochs_no_improve = 0
    best_model_path = "best_bertweet_model.pt"

    for epoch in range(epochs):
        # === Training ===
        model.train()
        loop = tqdm(train_loader, desc=f"Epoch {epoch}", leave=True)
        for batch in loop:
            optimizer.zero_grad()
            batch = {k: v.to(device) for k, v in batch.items()}
            outputs = model(**batch)
            loss = outputs.loss
            loss.backward()
            optimizer.step()
            scheduler.step()

            loop.set_postfix(loss=loss.item())

        # === Validation ===
        model.eval()
        val_preds = []
        val_true = []
        with torch.no_grad():
            for batch in val_loader:
                batch = {k: v.to(device) for k, v in batch.items()}
                outputs = model(**batch)
                logits = outputs.logits
                val_preds.extend(torch.argmax(logits, axis=-1).cpu().numpy())
                val_true.extend(batch["labels"].cpu().numpy())

        val_accuracy = accuracy_score(val_true, val_preds)
        print(f"Epoch {epoch} Validation Accuracy: {val_accuracy:.4f}")
```

```
# Early Stopping
if val_accuracy > best_val_accuracy:
    best_val_accuracy = val_accuracy
    epochs_no_improve = 0
    torch.save(model.state_dict(), best_model_path)
else:
    epochs_no_improve += 1
    if epochs_no_improve >= early_stop_patience:
        print("Early stopping triggered.")
        break

# === Load Best Model & Predict Test Set ===
model.load_state_dict(torch.load(best_model_path))
model.eval()
test_preds = []
with torch.no_grad():
    for batch in test_loader:
        batch = [k.v.to(device) for k, v in batch.items()]
        outputs = model(**batch)
        logits = outputs.logits
        test_preds.extend(torch.argmax(logits, axis=1).cpu().numpy())

return test_preds

#Run Final Training + Make Submission

final_test_preds = train_final_model_and_predict(
    train_texts=train_df["Text"].tolist(),
    train_labels=train_df["Category"].tolist(),
    test_texts=test_df["Text"].tolist(),
    lr=2e-5,      # Try 1e-5, 3e-5, or 2e-5
    epochs=5,     # Increase if you have time and compute resources
    max_length=128,
    early_stop_patience=2
)

# Prepare submission DataFrame
submission = pd.DataFrame({
    "ID": test_df["ID"],
    "Prediction": final_test_preds
})

submission_filename = "submission6.csv"
submission.to_csv(submission_filename, index=False)
print(f"Created {submission_filename}")

# Download if in Colab
files.download(submission_filename)

#####
# Model 7: BERTweet for Sentiment Classification with Scheduler & Early Stopping and 5-fold CV with 3 epochs each fold
#####

#Set Random Seed for Reproducibility
def set_seed(seed_value=42):
    random.seed(seed_value)
    np.random.seed(seed_value)
    torch.manual_seed(seed_value)
    torch.cuda.manual_seed_all(seed_value)

set_seed(42)

#Basic Preprocessing Function
def basic_preprocess(text):
    """
    Example function that removes URLs, user mentions,
    and repetitive punctuation. Adjust to your needs.
    """
    # Remove URLs
    text = re.sub(r"http[S+|www\S+]", "", text)
    # Remove user mentions
    text = re.sub(r"@[\w+]", "", text)
    # Reduce repeated punctuation (e.g. "!!!!" -> "!")
    text = re.sub(r"([!?,])\1+", r"\1", text)
    # Strip leading/trailing whitespace
    text = text.strip()
    return text

#Create a Custom Dataset Class
class BERTweetDataset(Dataset):
    def __init__(self, encodings, labels=None):
        self.encodings = encodings
        self.labels = labels

    def __len__(self):
        return len(self.encodings["input_ids"])

    def __getitem__(self, idx):
        item = {key: torch.tensor(val[idx]) for key, val in self.encodings.items()}
        if self.labels is not None:
            item["labels"] = torch.tensor(self.labels[idx])
        return item

#Define a Function to Train & Validate a Single Fold
def train_one_fold(
    model,
    train_loader,
    val_loader,
    device,
    epochs=3,
    lr=2e-5,
    early_stop_patience=2
):
    """
    Train the given model for a single fold with early stopping.
    Returns the best model (state_dict) and the best validation accuracy found.
    """
    optimizer = AdamW(model.parameters(), lr=lr)
```

```
num_training_steps = epochs * len(train_loader)
scheduler = get_linear_schedule_with_warmup(
    optimizer,
    num_warmup_steps=0,
    num_training_steps=num_training_steps
)

best_val_accuracy = 0.0
best_state_dict = None
epochs_no_improve = 0

for epoch in range(epochs):
    # --- TRAINING LOOP ---
    model.train()
    loop = tqdm(train_loader, desc=f"Epoch {epoch}", leave=False)
    for batch in loop:
        batch = {k: v.to(device) for k, v in batch.items()}
        optimizer.zero_grad()
        outputs = model(**batch)
        loss = outputs.loss
        loss.backward()
        optimizer.step()
        scheduler.step()

        loop.set_postfix(loss=loss.item())

    # --- VALIDATION LOOP ---
    model.eval()
    val_preds = []
    val_true = []
    with torch.no_grad():
        for batch in val_loader:
            batch = {k: v.to(device) for k, v in batch.items()}
            outputs = model(**batch)
            logits = outputs.logits
            val_preds.extend(torch.argmax(logits, axis=1).cpu().numpy())
            val_true.extend(batch["labels"].cpu().numpy())

    val_accuracy = accuracy_score(val_true, val_preds)

    # Check if this is the best model so far
    if val_accuracy > best_val_accuracy:
        best_val_accuracy = val_accuracy
        best_state_dict = model.state_dict() # snapshot
        epochs_no_improve = 0
    else:
        epochs_no_improve += 1
        if epochs_no_improve >= early_stop_patience:
            print(f"Early stopping triggered at epoch {epoch}")
            break

    return best_state_dict, best_val_accuracy

#K-Fold Cross Validation Routine
def k_fold_cv_bertweet(
    texts,
    labels,
    num_labels=4,
    n_splits=5,
    epochs=3,
    lr=2e-5,
    batch_size=16,
    max_length=128,
    early_stop_patience=2
):
    """
    Perform k-fold cross-validation using BERTweet.
    Returns the average validation accuracy across folds.
    """
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    tokenizer = AutoTokenizer.from_pretrained("vinai/bertweet-base")

    skf = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=42)
    fold_accuracies = []

    for fold_index, (train_idx, val_idx) in enumerate(skf.split(texts, labels)):
        print(f"--- Fold {fold_index+1}/{n_splits} ---")

        # Split data
        train_texts = [texts[i] for i in train_idx]
        train_lbls = [labels[i] for i in train_idx]
        val_texts = [texts[i] for i in val_idx]
        val_lbls = [labels[i] for i in val_idx]

        # Tokenize
        train_encodings = tokenizer(train_texts, truncation=True, padding=True, max_length=max_length)
        val_encodings = tokenizer(val_texts, truncation=True, padding=True, max_length=max_length)

        # Create datasets
        train_dataset = BERTweetDataset(train_encodings, train_lbls)
        val_dataset = BERTweetDataset(val_encodings, val_lbls)

        # Create DataLoaders
        train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
        val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)

        # Load model
        model = AutoModelForSequenceClassification.from_pretrained("vinai/bertweet-base", num_labels=num_labels)
        model.to(device)

        # Train & Evaluate on this fold
        best_state, best_acc = train_one_fold(
            model=model,
            train_loader=train_loader,
            val_loader=val_loader,
            device=device,
            epochs=epochs,
            lr=lr,
            early_stop_patience=early_stop_patience
        )
    
```

```
fold_accuracies.append(best_acc)
print(f"Fold {fold_index+1} best val accuracy = {best_acc:.4f}")

avg_accuracy = np.mean(fold_accuracies)
print(f"n=== Average {n_splits}-Fold Validation Accuracy: {avg_accuracy:.4f} ===")
return avg_accuracy

#Run K-Fold CV (to estimate performance & tune hyperparams) - 5-fold CV with 3 epochs each fold
all_texts = train_df["Text"].tolist()
all_labels = train_df["Category"].tolist()

cv_accuracy = k_fold_cv_bertweet(
    texts=all_texts,
    labels=all_labels,
    num_labels=4,
    n_splits=5,
    epochs=3,
    lr=2e-5,
    batch_size=16,
    max_length=128,
    early_stop_patience=2
)

#Train Final Model on Full Data & Predict Test
def train_final_model_and_predict(
    train_texts,
    train_labels,
    test_texts,
    lr=2e-5,
    epochs=5,
    max_length=128,
    early_stop_patience=2,
    batch_size=16
):
    """
    Trains a final BERTweet model on ALL training data, using a
    standard train/validation split to monitor early stopping.
    Returns test set predictions.
    """
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    tokenizer = AutoTokenizer.from_pretrained("vinai/bertweet-base")

    tr_texts, val_texts, tr_labels, val_labels = train_test_split(
        train_texts,
        train_labels,
        test_size=0.1,
        random_state=42,
        stratify=train_labels
    )

    # Tokenize
    train_encodings = tokenizer(tr_texts, truncation=True, padding=True, max_length=max_length)
    val_encodings = tokenizer(val_texts, truncation=True, padding=True, max_length=max_length)
    test_encodings = tokenizer(test_texts, truncation=True, padding=True, max_length=max_length)

    # Create datasets
    train_dataset = BERTweetDataset(train_encodings, tr_labels)
    val_dataset = BERTweetDataset(val_encodings, val_labels)
    test_dataset = BERTweetDataset(test_encodings)

    # Create loaders
    train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
    val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)
    test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)

    # Initialize model
    model = AutoModelForSequenceClassification.from_pretrained("vinai/bertweet-base", num_labels=4)
    model.to(device)

    # Optimizer & Scheduler
    optimizer = AdamW(model.parameters(), lr=lr)
    num_training_steps = epochs * len(train_loader)
    scheduler = get_linear_schedule_with_warmup(
        optimizer,
        num_warmup_steps=0,
        num_training_steps=num_training_steps
    )

    # Early Stopping Tracking
    best_val_accuracy = 0
    epochs_no_improve = 0
    best_model_path = "best_bertweet_model.pt"

    for epoch in range(epochs):
        model.train()
        loop = tqdm(train_loader, desc=f"Epoch {epoch}", leave=True)
        for batch in loop:
            optimizer.zero_grad()
            batch = {k: v.to(device) for k, v in batch.items()}
            outputs = model(**batch)
            loss = outputs.loss
            loss.backward()
            optimizer.step()
            scheduler.step()

            loop.set_postfix(loss=loss.item())

        # Validation step
        model.eval()
        val_preds = []
        val_true = []
        with torch.no_grad():
            for batch in val_loader:
                batch = {k: v.to(device) for k, v in batch.items()}
                outputs = model(**batch)
                logits = outputs.logits
                val_preds.extend(torch.argmax(logits, axis=-1).cpu().numpy())
                val_true.extend(batch["labels"].cpu().numpy())
```

```
val_accuracy = accuracy_score(val_true, val_preds)
print(f"[Epoch {epoch}] Validation Accuracy: {val_accuracy:.4f}")

if val_accuracy > best_val_accuracy:
    best_val_accuracy = val_accuracy
    epochs_no_improve = 0
    torch.save(model.state_dict(), best_model_path)
else:
    epochs_no_improve += 1
    if epochs_no_improve >= early_stop_patience:
        print("Early stopping triggered.")
        break

# Load best model
model.load_state_dict(torch.load(best_model_path))
model.eval()

# Predict test set
test_preds = []
with torch.no_grad():
    for batch in test_loader:
        batch = [k: v.to(device) for k, v in batch.items()]
        outputs = model(**batch)
        logits = outputs.logits
        test_preds.extend(torch.argmax(logits, axis=1).cpu().numpy())

return test_preds

#Final Training + Submission
final_preds = train_final_model_and_predict(
    train_texts=train_dff["Text"].tolist(),
    train_labels=train_dff["Category"].tolist(),
    test_texts=test_dff["Text"].tolist(),
    lr=2e-5,      # tuned from k-fold CV
    epochs=5,     # can adjust
    max_length=128,
    early_stop_patience=2,
    batch_size=16
)

# Build submission DataFrame
submission = pd.DataFrame({
    "ID": test_dff["ID"],
    "Prediction": final_preds
})
submission_filename = "submission7.csv"
submission.to_csv(submission_filename, index=False)
print(f"Created {submission_filename}")

# Download if using Colab
files.download(submission_filename)

# =====
# Line chart - model runtime (wrapped model names)
# =====

# 1. Load the Excel sheet -----
file_path = '/content/NLP using BERT Sentiment Analysis_Submission Scoreboard.xlsx'
df_full = pd.read_excel(file_path, sheet_name=0)

# 2. Robust column detection -----
def find_col(patterns):
    for patt in patterns:
        match = [c for c in df_full.columns if re.search(patt, c, flags=re.I)]
        if match:
            return match[0]
    raise KeyError(patterns)

# now includes plain 'Model' in addition to 'Model type' or 'Model #'
model_col = find_col([r'^model$', r'^model\s*type$', r'^model\s*#'])
runtime_col = find_col([r'runtime', r'time\s*taken'])

# 3. Trim & rename -----
df = df_full[[model_col, runtime_col]].copy()
df.columns = ['Model', 'RuntimeRaw']

# 4. Parse runtimes into seconds -----
def parse_runtime(x):
    if pd.isna(x):
        return np.nan
    if isinstance(x, (int, float)):
        return float(x)
    s = str(x).lower().strip()

    # HH:MM:SS or MM:SS
    if re.match(r'^\d{1,2}:\d{2}(\:\d{2})?$', s):
        return pd.to_timedelta('0:' + s if s.count(':') == 1 else s).total_seconds()

    # XmYs or Xms
    m = re.match(r'^(\d+)?s*m(?:in)?(\d+)?s*(?:ec)?$', s)
    if m and (m.group(1) or m.group(2)):
        minutes = int(m.group(1) or 0)
        seconds = int(m.group(2) or 0)
        return minutes * 60 + seconds

    # plain seconds e.g. "115 s"
    m = re.match(r'^(\d+)?s$', s)
    if m:
        return float(m.group(1))

    # fallback
    try:
        return pd.to_timedelta(s).total_seconds()
    except ValueError:
        return np.nan

dff['Seconds'] = dff['RuntimeRaw'].apply(parse_runtime)
df.dropna(subset=['Seconds'], inplace=True)
dff['Minutes'] = dff['Seconds'] / 60
```

```
# 5. Wrap model names (15-char lines) -----
wrapped = ['\n'.join(textwrap.wrap(str(m), 15)) for m in df['Model']]

# 6. Plot -----
fig, ax = plt.subplots(figsize=(12, 6))

ax.plot(wrapped, df['Minutes'],
        color='#4F81BD', markers='o', linewidth=2, markersize=8)
ax.fill_between(wrapped, df['Minutes'],
               color='#4F81BD', alpha=0.15)

# annotate each point (mm:ss)
for x, sec in zip(wrapped, df['Seconds']):
    mm = int(sec // 60)
    ss = int(sec % 60)
    ax.text(x, sec / 60 + 0.02 * df['Minutes'].max(),
           f'{mm}:{ss:02d}', ha='center', va='bottom', fontsize=9)

ax.set_ylabel('Runtime (minutes)')
ax.set_xlabel('Model')
ax.set_title('Model Training / Inference Runtime')
ax.grid(axis='y', linestyle='-', alpha=0.4)
ax.set_ylim(0, df['Minutes'].max() * 1.15)

plt.tight_layout()
plt.show()

# =====
# Single Model AUC-ROC Performance Visualization
# =====

# 1. Load Data -----
file_path = '/content/Binary Prediction of Rainfall Scores copy.xlsx'
df = pd.read_excel(file_path)

# Filter single models only
df['Model #'] = pd.to_numeric(df['Model #'], errors='coerce')
df_single = df[df['Model #'] <= 7].copy()

# Rename columns
df_single.columns = [col.strip() for col in df_single.columns]
df_single = df_single.rename(columns={
    'Validation AUC ROC': 'Validation',
    'Public Kaggle Score': 'Public',
    'Private Kaggle Score': 'Private'
})

# 2. Cozy Color Palette -----
colors = {'Validation': '#4F81BD', # Blue
          'Public': '#4BACC6', # Teal
          'Private': '#D38F97'} # Blush

# 3. Wrap model names for compact labels -----
wrapped_labels = ['\n'.join(textwrap.wrap(str(m), 15)) for m in df_single['Model']]

# 4. Plot Setup -----
x = np.arange(len(df_single))
width = 0.27

fig, ax = plt.subplots(figsize=(16, 6))

bars_val = ax.bar(x - width, df_single['Validation'], width, color=colors['Validation'], label='Validation AUC')
bars_pub = ax.bar(x, df_single['Public'], width, color=colors['Public'], label='Public Kaggle Score')
bars_pri = ax.bar(x + width, df_single['Private'], width, color=colors['Private'], label='Private Kaggle Score')

# Axes and titles
ax.set_xticks(x)
ax.set_xticklabels(wrapped_labels, ha='center', fontsize=10, fontweight='bold')
ax.set_ylabel('AUC-ROC Score', fontsize=14, fontweight='bold')
ax.set_xlabel('Model', fontsize=13, fontweight='bold')
ax.set_title('Single Model Performance (Validation vs Kaggle Scores)', fontsize=16, fontweight='bold')

# Y-axis zoom
lowest = df_single[['Validation', 'Public', 'Private']].min().min()
ax.set_ylim(max(0.84, lowest - 0.002), 0.922)
ax.grid(False)

# 5. Vertical value labels -----
def add_labels(bars):
    for bar in bars:
        h = bar.get_height()
        ax.text(bar.get_x() + bar.get_width()/2,
                h + 0.0005,
                f'{h:.4f}',
                ha='center', va='bottom',
                rotation=90, fontsize=9, fontweight='bold')

add_labels(bars_val)
add_labels(bars_pub)
add_labels(bars_pri)

# 6. Legend outside the plot -----
ax.legend(loc='center left', bbox_to_anchor=(1.02, 0.5), frameon=False, fontsize=11)

plt.tight_layout()
plt.show()
```